

高性能计算导论

第8讲：CUDA编程模型-2

翟季冬
计算机系

目录

- GPU 执行模型
- GPU 存储层次
- CUDA 程序优化
 - 数据传输优化
 - 全局内存优化
 - 共享内存优化

目录

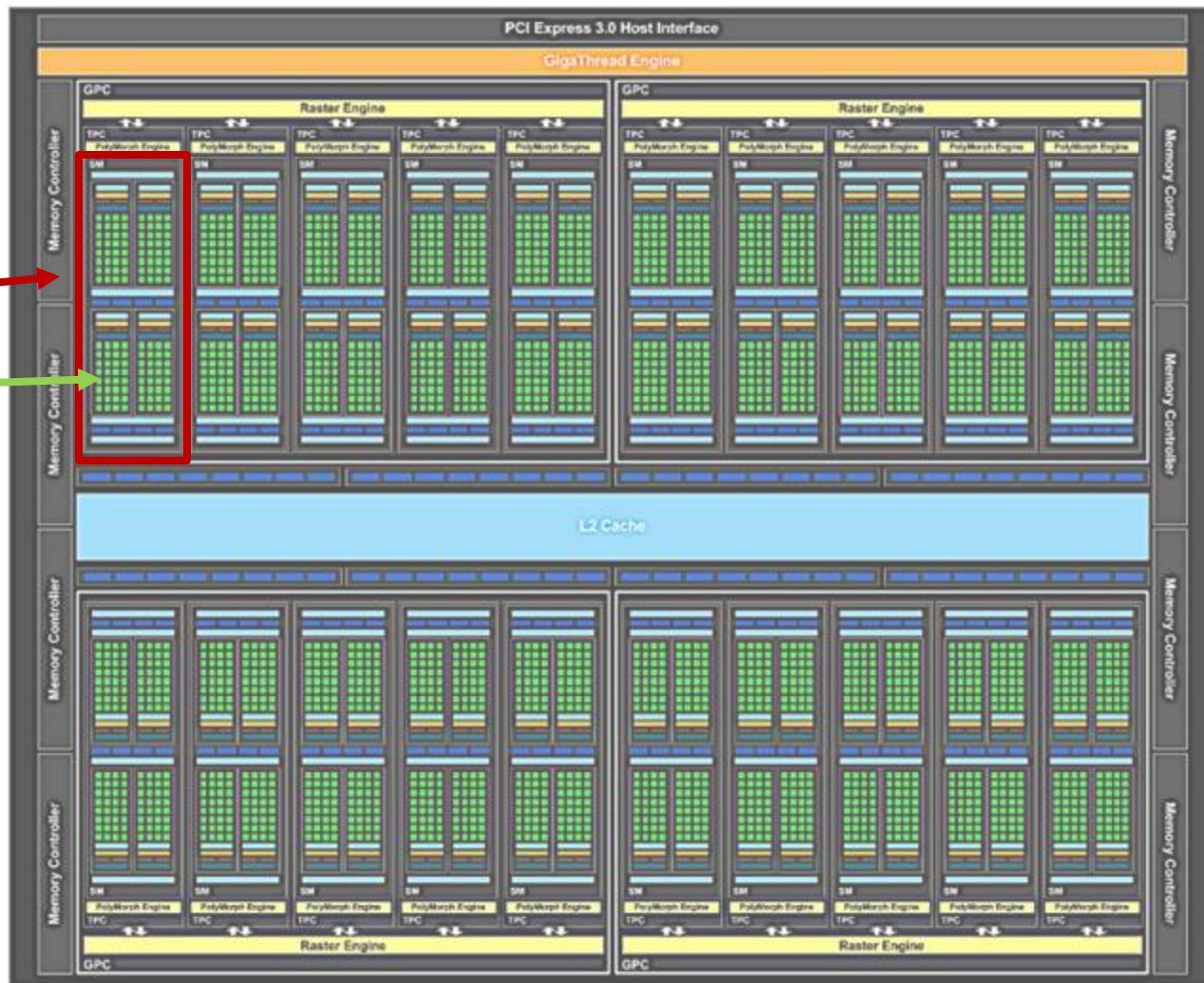
- GPU 执行模型
- GPU 存储层次
- CUDA 程序优化
 - 数据传输优化
 - 全局内存优化
 - 共享内存优化

GPU 执行模型

回顾 GPU 硬件组织

■ GPU 计算硬件组织:

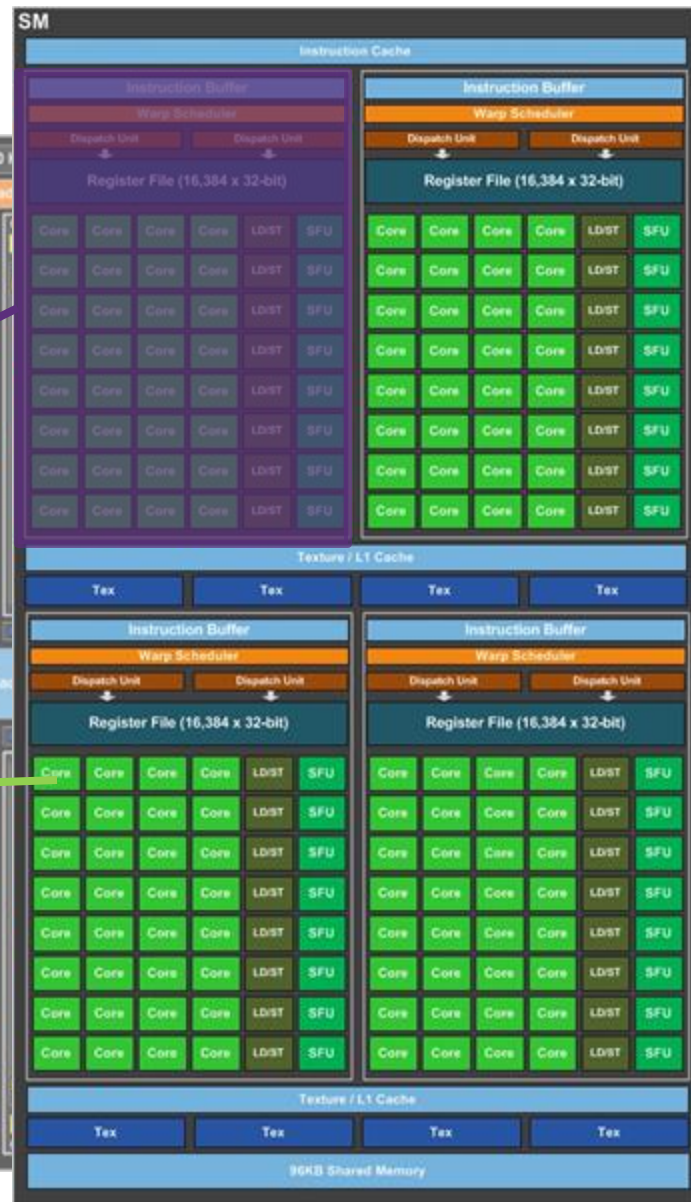
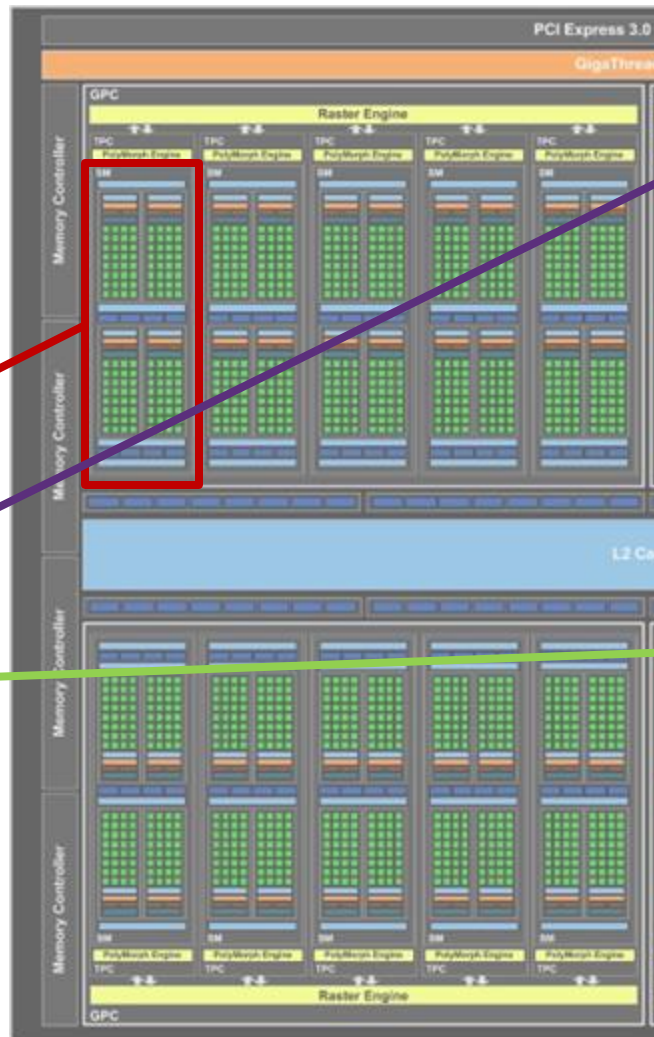
- GPU
- Streaming Multiprocessor (SM)
- Core



回顾 GPU 硬件组织

■ GPU硬件层次

- GPU
- Streaming Multiprocessor (SM)
- Warp scheduler
- Core



回顾 CUDA 代码

■ CUDA 代码

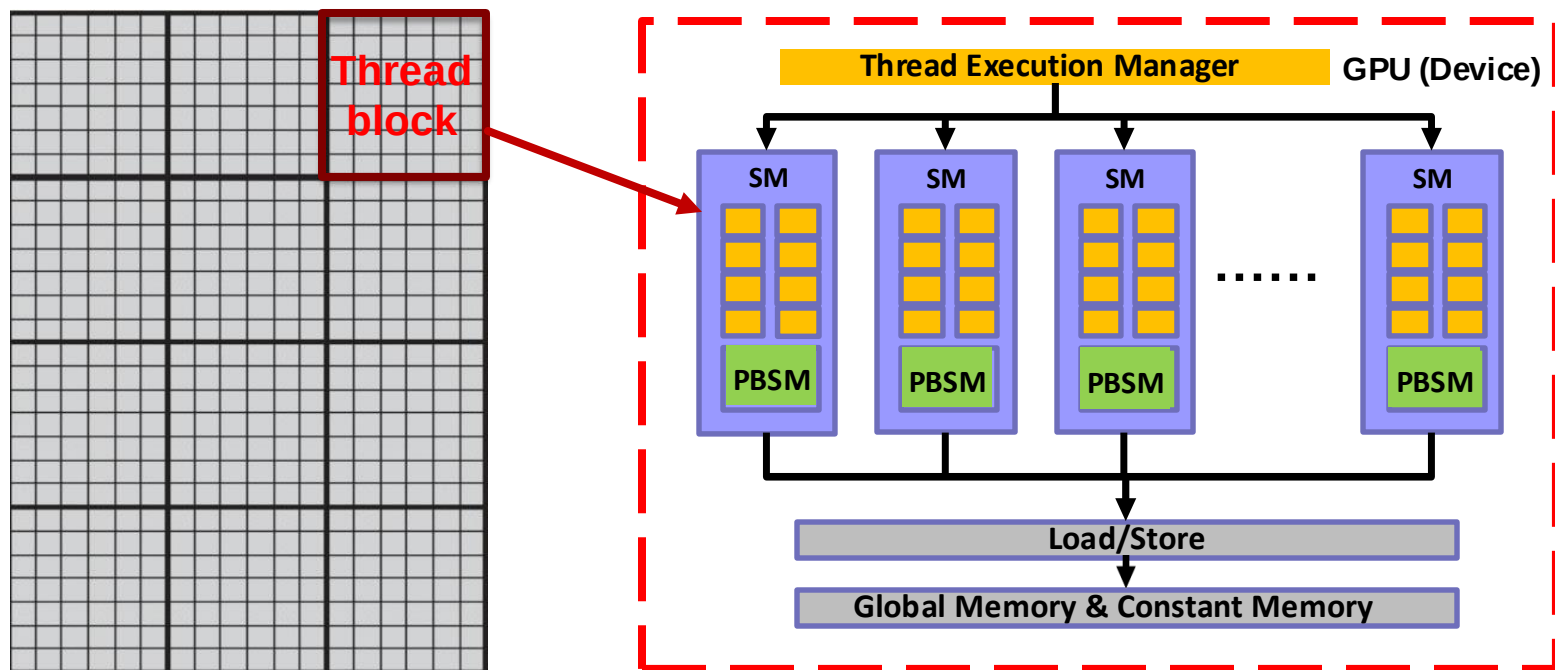
- CUDA 程序 = Host 程序 + Device 程序（也叫核函数 kernel Function）
- Grid → thread blocks → threads

```
__global__ void MatAdd(float **A, float **B, float **C)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    C[i][j] = A[i][j] + B[i][j];
}

int main()
{
    dim3 threadsPerBlock(16, 16);
    dim3 numBlocks(N/16, N/16);
    // 每个线程块 256 个线程, N*N/256 个线程块
    MatAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
}
```

CUDA 线程组织

- CUDA 的线程被组织成**线程块**
 - 同一个线程块内的**所有线程**：
 - ✓ 被分配到**同一个 SM** 上执行
 - ✓ 可以通过 SM 上**共享内存**进行线程间通信



线程块执行

- 线程块执行：
 - 线程块执行在 SM 上
 - 在一个 SM 上，如果有**可用的**硬件资源，SM 上可以同时执行**多个线程块**
 - 硬件资源：
 - **寄存器、共享内存**等
 - 编译时计算
 - 在一个 SM 上，如果**没有可用**的硬件资源
 - 必须等待上一个线程块执行完毕，才能执行下一个线程块执行
 - 由于线程块执行时间不确定，所有**线程块的执行顺序**没有保证

线程块是 GPU 调度的最小单元吗？

不是

Warp 概念

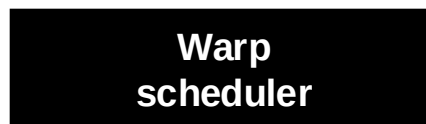
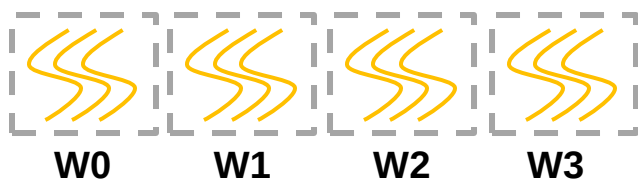
- Warp 定义：
 - 在 SM 上，每次调度一组线程 → warp
 - 一个 warp 包括 32 个线程
 - Warp 是 SM 上线程调度的最小单位
 - Warp 内的所有线程同时执行相同指令（SIMD）
- 换一种说法：
 - Warp 内的线程物理上被同时执行
 - 线程块内的线程逻辑上被同时执行
 - 由于 warp 之间执行顺序不固定，kernel 需要在线程块内同步线程
- 在 Pascal 和 Volta 架构上
 - 每个 SM 上最多 32 个线程块
 - 每个 SM 上最多 64 个 warp
 - 每个 SM 上最多 2048 个活跃线程（ $64 * 32$ ）

Warp 调度

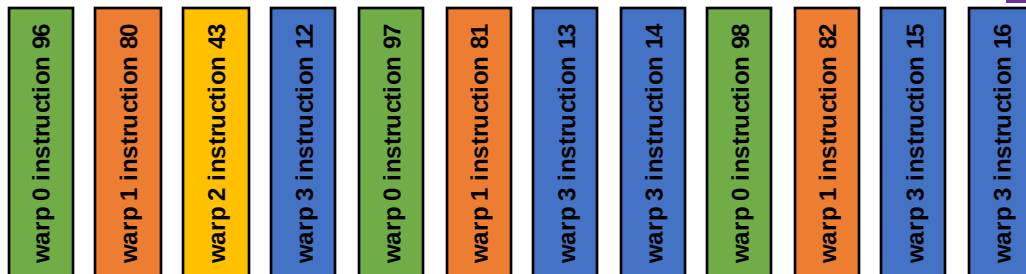
- Warp 调度器 (warp scheduler)
 - SM 上的硬件单元
 - 决定每个周期执行哪一个 warp 上的指令
- Warp 调度



- 一个 Warp 的下一条指令操作数就绪，就具备被调度执行的资格
- 一个 warp 出现内存等待 (memory stall) 时将被换出
- Warp 被调度执行时，warp 内的所有线程将执行相同指令 (SIMT)



不同于CPU的指令调度



时间

GPU 执行模型

软件视角



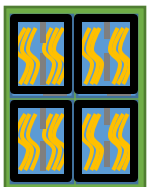
thread



warp



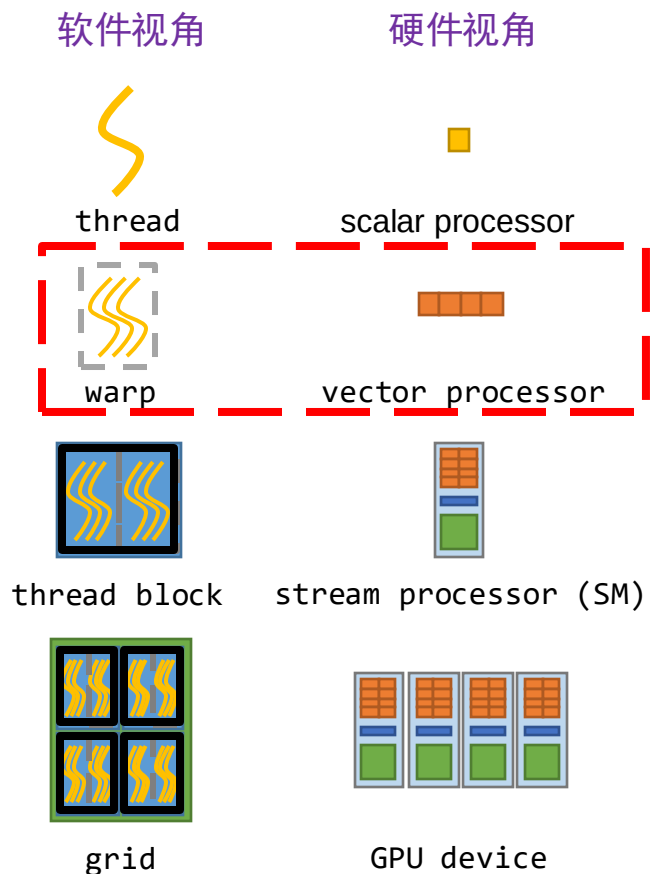
thread block



grid

- 每个线程可以理解为通过由一个**标量处理器**执行
- 每个 warp 通过一个**向量处理器**执行
- 每个线程块被分配到一个 **SM** 上执行
- 多个并发线程块可以被分配到**同一个 SM**
- 一个 kernel 由**一组线程块**执行

GPU 执行模型

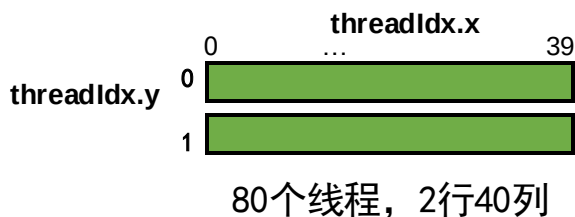


- 每个线程可以理解为通过由一个标量处理器执行
- 每个 warp 通过一个向量处理器执行
- 每个线程块被分配到一个 SM 上执行
- 多个并发线程块可以被分配到同一个 SM
- 一个 kernel 由一组线程块执行

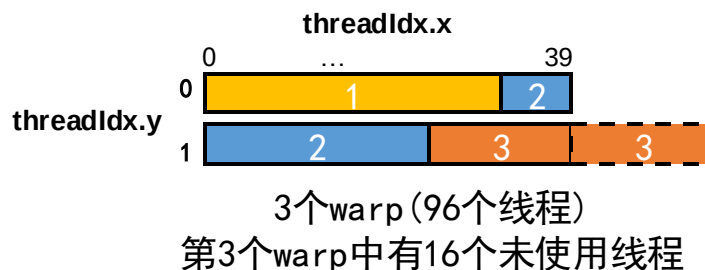
CUDA 线程组织

- CUDA 程序层次结构：
 - Grid → Thread blocks → Warp → Threads
 - Warp = 连续的 32 threads
- 线程块中的线程按顺序组织为 warp
 - 线程块中的维度只是逻辑概念
 - 线程块按维度展开后，每 32 个线程组织成一个 warp

```
dim3 gridSize (1, 1);  
dim3 blockSize (40, 2);  
kernel_tb_example <<<gridSize, blockSize>>>();
```



逻辑组织



物理组织

WARP DIVERGENCE -WARP发散

Warp 发散 - Warp Divergence

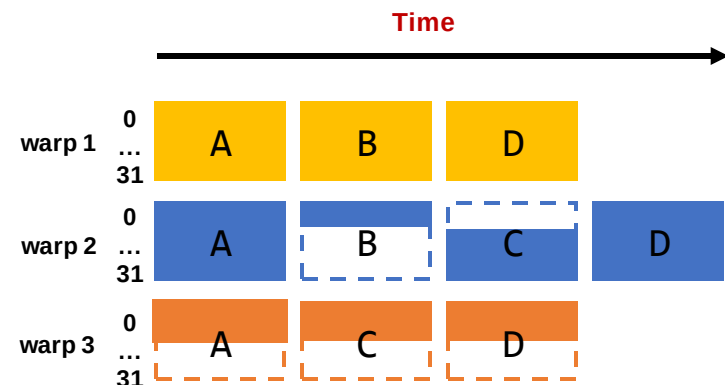
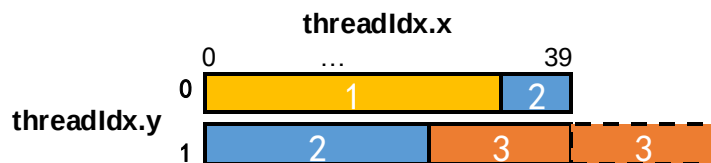
- Warp Divergence:
 - 同一个 warp 内不同线程执行不同程序分支 → 线程发散，严重影响性能
 - 线程发散 → 同一个 warp 内的不同执行部分会被顺序执行
 - 不同 warp 执行不同代码时，没有性能影响

```
if (threadIdx.x < 2) {  
    // A部分  
} else {  
    // B部分  
}
```


Warp 发散 – 复杂例子

```
__global__ void warp_example() {  
    A();  
    if (threadIdx.y == 0) {  
        B();  
    } else {  
        C();  
    }  
    D();  
}  
  
int main() {  
    dim3 gridSize (1, 1);  
    dim3 blockSize (40, 2);  
    warp_example <<<gridSize, blockSize>>>();  
}
```

- Warp 1 没有发散
- Warp 2 产生发散
- Warp 3 没有发散、
没有用满



避免 warp 发散

- Warp 发散的例子：分支粒度 < warp size

```
if (threadIdx.x > 2) {  
    // ...  
} else {  
    // ...  
}
```

- 无 warp 发散的例子：以 warp size 作为分支粒度

```
if (threadIdx.x / WARP_SIZE > 2) {  
    // ...  
} else {  
    // ...  
}
```

- 避免 warp 发散：

- 按照 warp 选择合适的线程块大小，提供合适的分支粒度（warp size）
- 减少 warp 内的分支情况
- warp 之间的分支不影响性能，不需要考虑

GPU 存储层次

GPU 存储层次

- 寄存器 – register file
 - 每个线程可读写
 - 最低延迟、最高带宽
- 共享内存 - shared memory
 - 每个线程块可读写
 - 较低延迟、较高带宽
- 全局内存 - global memory
 - 每个核心可读写
 - 高延迟、低带宽
- 本地内存 - local memory
 - 每个线程可读写
 - 在全局内存上（global memory）
- 常量内存 - constant memory
 - 每个核心，只读



GPU 存储层次示例

■ 以 Tesla V100 为例

■ SM 数量

- 80 个

■ 寄存器

- 256KB/SM, 共 20MB

■ 共享内存/L1 cache

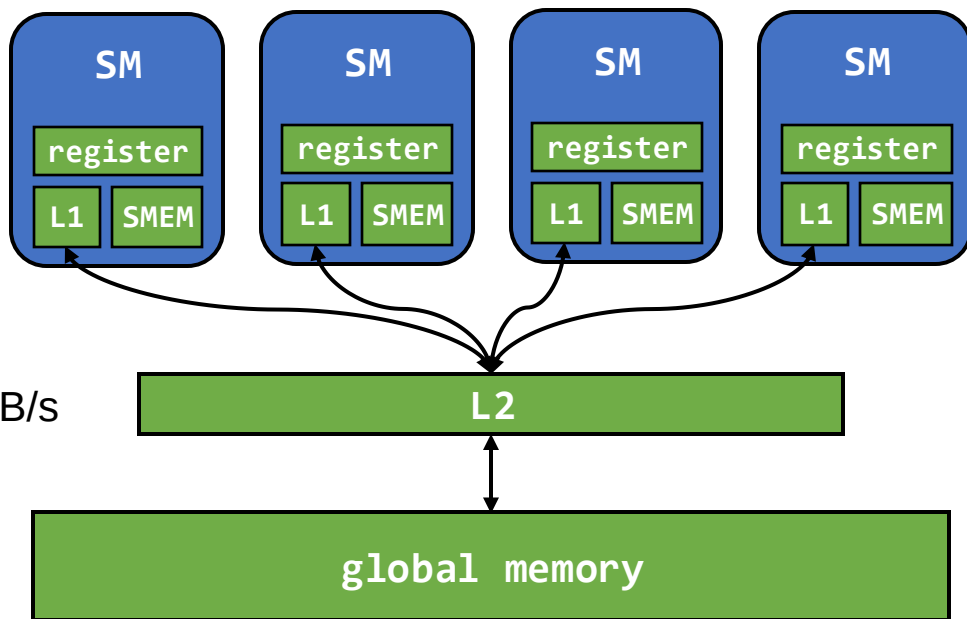
- 128KB/SM, 共 10MB
- 用户可以配置两者的比例, $\sim 14\text{TB/s}$

■ L2 cache

- 6MB
- read: $\sim 2.5\text{TB/s}$ write: $\sim 1.6\text{TB/s}$

■ 全局内存

- 16/32 GB HBM2
- $\sim 900\text{GB/s}$



Cache

- 在 GPU 中，cache分 L1 cache 和 L2 cache
- L1 cache
 - L1 cache 是每个 SM 上的物理资源
 - 无法跨 SM 复用数据
- L2 cache
 - L2 cache 是设备全局共享的物理资源
 - 可以跨 SM 复用数据
- 当访问一个内存地址时
 - 首先查询 L1，若数据在 L1 中，则直接访问
 - 否则查询 L2，若数据在 L2 中直接访问
 - 若 L1、L2 均未命中，再访问 DRAM

寄存器

- 寄存器：
 - 性能好、容量有限
 - V100 上每个 SM 有 256KB 的寄存器文件
 - 每个线程最多使用 255 个 4-bytes 寄存器
- 寄存器读写不会带来额外开销，但以下情况除外：
 - 寄存器间存在写后读（read-after-write）的依赖
 - 寄存器发生冲突
- Register 溢出
 - 每个线程最大使用寄存器个数为 255 个
 - 当寄存器不足的时候，会溢出到本地内存（local memory）
 - 本地内存最大限制为每个线程 512KB

本地内存 - Local memory

- 本地内存：
 - 每个线程一块私有内存，存放线程私有数据
 - 逻辑概念，实现在全局内存上、性能和全局内存一致
 - 注意：不是一块单独硬件
- 本地内存使用：
 - 在内核定义本地变量时，如果寄存器可用，会优先使用寄存器
 - 当线程寄存器不足时，私有数据会溢出到本地内存
 - 在编译阶段根据情况，计算本地内存的使用

```
__global__ void example(float *dst, float *src) {  
    int i, j, k;  
    float buf_a[4], buf_b[4];  
}
```


共享内存 – Shared memory

- 共享内存：
 - 本质上是一种手动 cache
 - 和 L1 共用同一块硬件资源
 - 二者之间可以灵活进行配置
- 作用域：
 - 线程块中所有线程
 - 同一个线程块中的线程可以通过共享内存进行通信
 - 不同线程块中的线程只能通过全局内存进行通信
- 容量和 GPU 型号相关
 - P100：专用的 64KB 共享内存，不可配置
 - V100：和 L1 共用同一块硬件资源，共 128KB，最高可配置 96KB

共享内存的定义 - 静态定义

- 共享内存支持两种定义方式: 静态定义与动态定义
- 静态定义在核心代码中显示指定共享内存的大小
 - 要求长度必须是常量

为什么?

```
__global__ void example_shared_static() {  
    constexpr int shared_size = 32;  
    __shared__ float smem[shared_size];  
}
```

- 当长度为变量时, 会发生编译错误

```
__global__ void example_shared_static_error() {  
    int shared_size = 32;  
    __shared__ float smem[shared_size];  
}
```

共享内存的定义 - 动态定义

- 动态定义需要在**内核调用时**指定共享内存大小

```
__global__ void example_shared_dynamic() {  
    // shared_size = 32  
    extern __shared__ float smem[];  
}  
  
int main() {  
    example_shared_dynamic<<<gridSize, blockSize, 32>>>();  
}
```

- 当需要指定多个共享内存空间时，直接定义会导致两个地址相同

```
__global__ void example_shared_multi_dynamic_wrong() {  
    extern __shared__ float smem_a[];  
    extern __shared__ float smem_b[];  
}
```



- 需要通过**手动指定偏移量**，实现多个共享空间的定义

```
__global__ void example_shared_multi_dynamic() {  
    extern __shared__ float smem[];  
    float *smem_a = smem;  
    float *smem_b = smem + shared_size_a;  
}
```

全局内存 – Global memory

- 全局内存
 - 需要在 host 端声明
 - 通过 `cudaMalloc` 分配内存
 - 通过 `cudaMemcpy` 将数据在 host 和 device 之间传输
- 示例：
 - 数据通过 `cudaMemcpyH2D` 从 host 端拷贝到 device 端
 - 计算核心执行完后，通过 `cudaMemcpyD2H` 拷贝回 host 端

```
float *host_ptr = 0, *dev_ptr = 0;
host_ptr = (float *)malloc(size);
cudaMalloc((void **)&dev_ptr, size);
// ...
cudaMemcpy(dev_ptr, host_ptr, size, cudaMemcpyHostToDevice);
// kernel
cudaMemcpy(host_ptr, dev_ptr, size, cudaMemcpyDeviceToHost);
// ...
free(host_ptr);
cudaFree(dev_ptr);
```

常量内存 – Constant memory

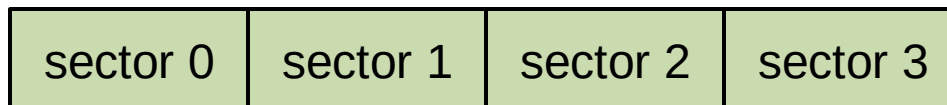
- 常量内存
 - 在内核内只读，共 64KB
 - 非片上内存，和全局内存有类似的用法和作用域
 - 通过 `cudaMemcpyToSymbol` 将数据拷贝到常量内存
 - 常量内存的 cache 和全局内存分开，两者互不影响
 - 可以通过常量内存优化访存的局部性

```
__constant__ int cmem[cmem_size];  
__global__ void example_constant() {  
    int buf = cmem[threadIdx.x];  
}  
int main() {  
    int data[cmem_size];  
    // prepare data  
    cudaMemcpyToSymbol(cmem, data, cmem_size * sizeof(int));  
    example_constant<<<gridSize, blockSize>>>();  
}
```

Sector 概念

- Sector: 在 DRAM、L2 cache、L1 cache 之间传输数据的**最小单元**
 - 1 sector = 32 Bytes
 - 对1个 sector 的访问被称为一个 Transaction
- 注意:
 - 在 Maxwell、Pascal、Volta 等新架构中，最小单元为 32Byte
 - 再之前的架构中最小访问粒度和架构、cache、类型相关，请查阅文档
 - 程序访存的 transaction 个数直接**影响访存性能**
- 1 cache line = 128 Bytes = 4 sectors
 - 针对 cache 优化的最小粒度是 cache line

128B 对齐



128B cache line

GPU 存储层次

Memory	On chip ?	Cached ?	Access	Scope	Lifetime
Register	on	n/a	r/w	thread	thread
Local	off	yes	r/w	thread	thread
Shared	on	n/a	r/w	block	block
Global	off	yes	r/w	all + host	host 分配释放
Constant	off	yes	r	all + host	host 分配释放
Texture	off	yes	r	all + host	host 分配释放

CUDA 程序优化

异构系统：设备性能差异

- 计算和访存**峰值性能差异**
 - Volta 上 FP32 的峰值性能为 **15.7T FLOPS**
 - 显存带宽为 900GB/s, 相当于每秒传输 **225G** 个 FP32 浮点数 (900/4)
 - **计算和全局访存峰值性能差为 ~70倍** (15.7T/225G)
- 功能部件之间**执行性能差异**
 - 计算指令耗时为 **1~4** 个时钟周期
 - 对全局内存的访问, 每次读写需要 **~400** 个时钟周期
 - 共享内存的访问需要 **1~20** 时钟周期
 - 计算访存性能比最多 **~100倍**
- 不同设备的**带宽差异**
 - CPU 和 GPU之间带宽 **~16GB/s**
 - 全局内存带宽 **~900GB/s**
 - 共享内存带宽 **~14TB/s**

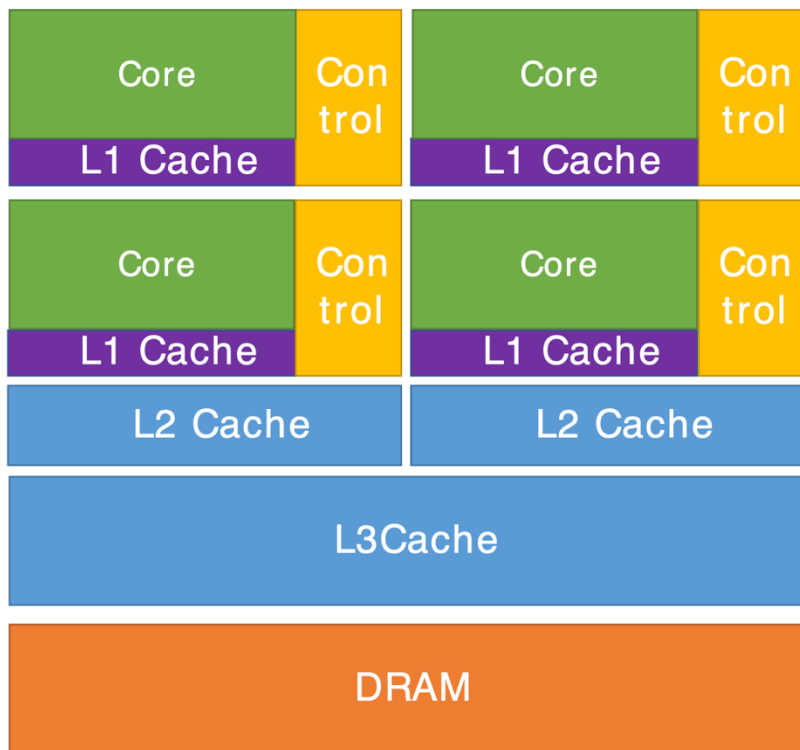
如何充分发挥不同硬件的性能是 CUDA 程序性能优化的关键之一

CUDA 程序典型优化

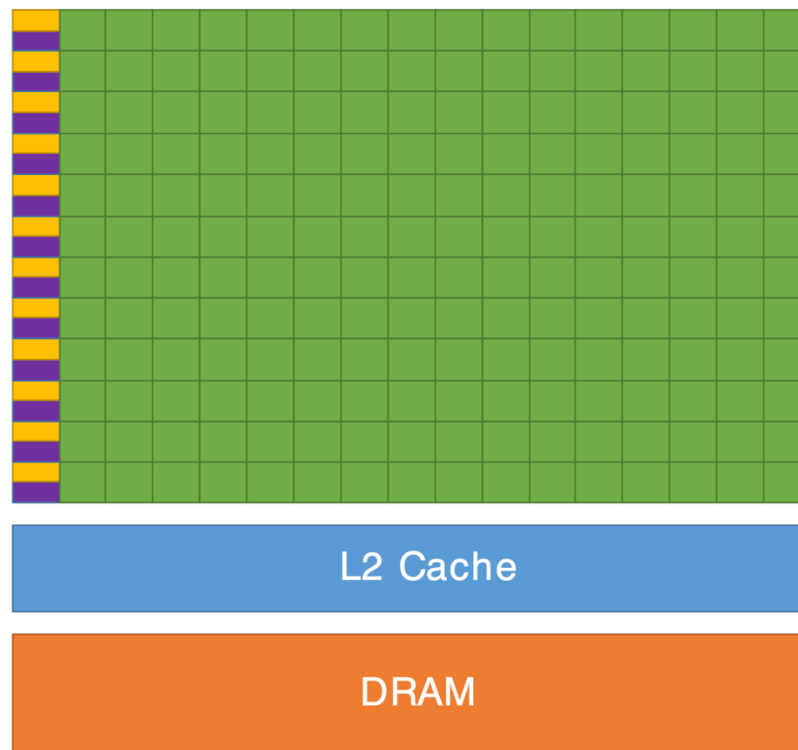
- 典型优化方法
 - 尽量减少 CPU 与 GPU 之间的通信次数与通信量
 - 尽量减少全局内存的访问，利用共享内存和寄存器缓存数据
 - 尽量对高速存储内的数据进行重用
 - 通过预取的方式重叠计算与访存

GPU 架构特点

- GPU：大量计算核心
- 典型 CUDA 程序：创建大量并发线程发挥 GPU 吞吐能力



CPU



GPU

GPU 占用率 - Occupancy

- $\text{Occupancy} = \text{SM 上运行的线程数} / \text{SM 最大支持线程数}$
 - 可以使用 nvprof 工具查看占用率
- 更高的占用率意味着
 - 同一个 SM 上有更多的 warp
 - 便于 warp scheduler 进行调度
 - 更好的掩盖指令延迟，提高流水线的利用率
- 想要达到更高的占用率
 - 需要在 SM 上有能够调度足够多的线程
 - 同时每个线程占用资源足够低

以寄存器为例分析

- 寄存器限制：
 - 每个 SM 上的寄存器资源 256KB
 - 每个线程最多使用 255个 4Byte 寄存器
 - 一个 SM 上的线程数越多，每个线程可用的寄存器数量就越少
 - 如果每个线程用满 255 个寄存器，那么 SM 上最多可以调度8个 warp
($256K/255*4*32$)

矛盾：

- 线程数太少 → 占用率低，对 warp 调度器不友好
 - 无法有效发挥GPU硬件利用率
 - 无法有效隐藏访存延迟
- 线程数太多
 - SM 资源（寄存器和共享内存）有限

因此，控制单个线程寄存器、共享内存等资源的使用，提高程序的 GPU 占用率是GPU程序优化的关键之一

循环展开

- 循环的分支会影响流水线的执行效率
 - 分支指令不进行计算或访存
 - 循环会降低寄存器利用率
- 通过循环展开减少分支
 - 右侧为手动展开与自动展开
- `#pragma unroll` 可以展开循环
 - 展开的结果和手动展开相同
 - `#pragma unroll 1`
 - 指示编译器不做任何循环展开

```
for (i = 0; i < n; i++) {  
    a = a + i;  
}
```

```
// 手动展开 4 次  
for (i = 0; i < n; i += 4)  
{  
    a = a + i;  
    a = a + i + 1;  
    a = a + i + 2;  
    a = a + i + 3;  
}
```

```
// 自动展开  
# pragma unroll 4  
for (int i = 0; i < n; ++  
i) {  
    a = a + i;  
}
```

原子操作-1

- 有时程序会需要多个线程更新共享或全局内存上相同变量

```
__shared__ int count;  
// ...  
if (...) {  
    count++;  
}
```

- **数据竞争**：多个线程同时更新同一个变量
- **解决方法**：使用 GPU 的原子操作，GPU原子操作支持以下操作
 - addition / subtraction
 - max / min
 - increment / decrement
 - compare-and-swap

原子操作-2

- 例子1: Histogram 统计图片的颜色分布, 每个线程负责一个位置

```
__global__ void hist(int *color, int *bin) {  
    int i = threadIdx.x + blockDim.x * blockIdx.x;  
    int c = colors[i];  
    atomicAdd(&bin[c], 1);  
}
```

- 例子2: 全局 max, 每个线程负责其中一个元素

```
__global__ void max_value(int *values, int *global_max) {  
    int i = threadIdx.x + blockDim.x * blockIdx.x;  
    int val = values[i];  
    atomicMax(global_max, val);  
}
```

- 原子操作的性能相比于普通的操作有性能降低
- 特别是对共享内存上的数据, 性能影响大
- 对全局内存上的数据, 会降低少量性能

CUDA程序优化

- 数据传输优化
- 全局内存优化
- 共享内存优化

数据传输优化

Host 和 Device 之间数据传输

- CPU 与 GPU 之间的带宽比全局内存的访问带宽低得多
 - 15.8GB/s (PCI-e x16 Gen 3) vs. 900GB/s peak (V100)

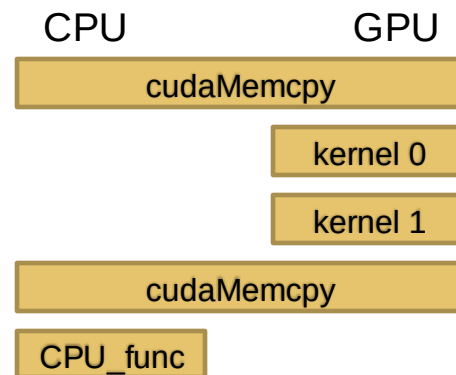
典型优化策略：

- 最小数据传输
 - 中间数据应当尽量在 GPU 上被创建、使用和销毁，而不要被传回 CPU
- 合并数据传输
 - 将多次小的传输合并为一次大的传输比单独进行传输要性能好
- 异步数据传输
 - 重叠计算时间与通信时间
- 使用 固定内存 技术获得更高的带宽

数据传输

1. `cudaMemcpy` 是同步执行
2. 从相同线程发起的 kernel 被顺序执行

```
cudaMalloc(&dev1, size);  
double *host1 = (double *)malloc(&host1, size);  
// ...  
// cudaMemcpy blocks until copy is completed  
cudaMemcpy(dev1, host1, size, H2D);  
// two kernels are serialized and executed  
// on device  
kernel0<<<grid, block>>>( ..., dev0, ... );  
kernel1<<<grid, block>>>( ..., dev1, ... );  
// cudaMemcpy starts after kernels finish  
// and blocks until copy is completed  
cudaMemcpy(host4, dev4, size, D2H);  
CPU_func();  
// ...
```

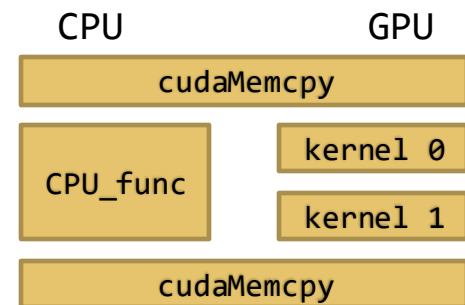


GPU 上计算要等待数据传输完，才可计算；CPU 计算也要等待数据传输完

优化-调整执行顺序

- 调整代码顺序
 - GPU 的 Kernel 和 CPU 代码是异步执行的
 - 但 Kernel 0 和 Kernel 1 还是顺序执行（发给同一设备的 kernel）

```
cudaMalloc(&dev1, size);  
double *host1 = (double *)malloc(&host1, size);  
// ...  
cudaMemcpy(dev1, host1, size, H2D);  
kernel0<<<grid, block>>>( ..., dev0, ... );  
kernel1<<<grid, block>>>( ..., dev1, ... );  
CPU_method();  
cudaMemcpy(host4, dev4, size, D2H);  
// ...
```



异步执行

■ 异步执行：

- 为了方便在 CPU 和 GPU 上并行执行，一些函数被设计成异步的
- 异步：在设备完成请求前把控制权返回给 Host 线程

■ 异步函数：

- Kernel launch
- 异步数据传输：
 - `cudaMemcpyAsync`、`cudaMemsetAsync`

■ 异步数据传输：

- Host 和 Device 之间的数据传输能够立即返回控制到 CPU
- `cudaMemcpyAsync(dst, src, size, dir, stream)`
- 可以实现 CPU 计算和数据传输的重叠

```
cudaMemcpyAsync(a_d, a_h, size, cudaMemcpyHostToDevice, 0);  
kernel<<<grid, block>>>(a_d);  
CPU_func();
```

Stream 概念

- Host 发给同一设备上的一系列操作**按照顺序执行**（默认一个 stream）

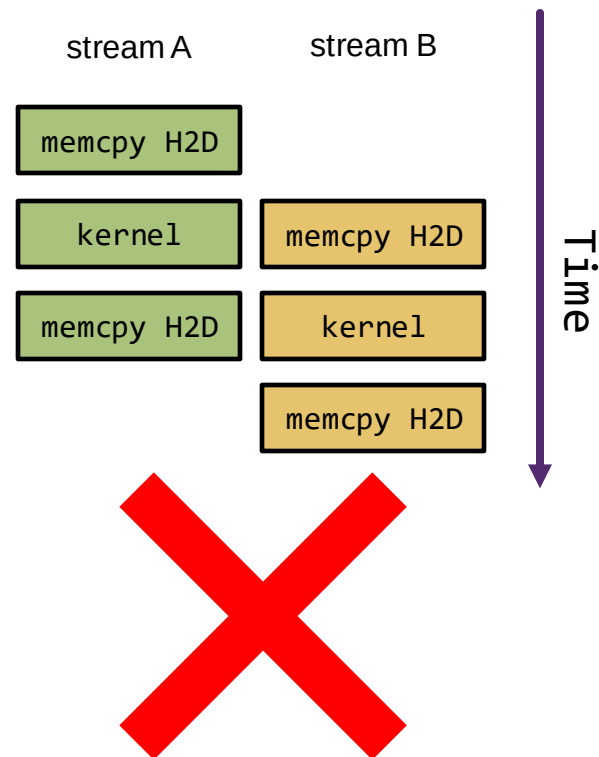
```
kernelFunc<<< nB, nT, nS, Sid >>> (...); // Sid is the streamID
```

- **Stream概念:**
 - 不同 stream 中的操作会被**交错执行**，可能的话会被**同时执行**
 - Stream 之间的**执行顺序不确定**
 - 不进行任何设置会使用默认 stream（stream 0）
- 当设备**能力允许**时（依赖于底层硬件实现）：
 - 重叠数据传输和内核执行
 - 并发的内核执行
 - 并发的数据传输（H2D 和 D2H）

通过 Stream 并发

- 实现并发：
 - 多个 stream

```
cudaStream_t stream[2];
cudaStreamCreate(&stream[0]);
cudaStreamCreate(&stream[1]);
cudaMallocHost(&hostPtr, 2 * size); // page
locked mem)
for (int i = 0; i < 2; ++i) {
    // memcpy
    cudaMemcpy(/*...*/, cudaMemcpyHostToDevice, stream[i]);
    kernel<<<100,512,0,stream[i]>>>(/*...*/);
    cudaMemcpy(/*...*/, cudaMemcpyDeviceToHost, stream[i]);
}
cudaStreamDestroy(stream[0]);
cudaStreamDestroy(stream[1]);
```

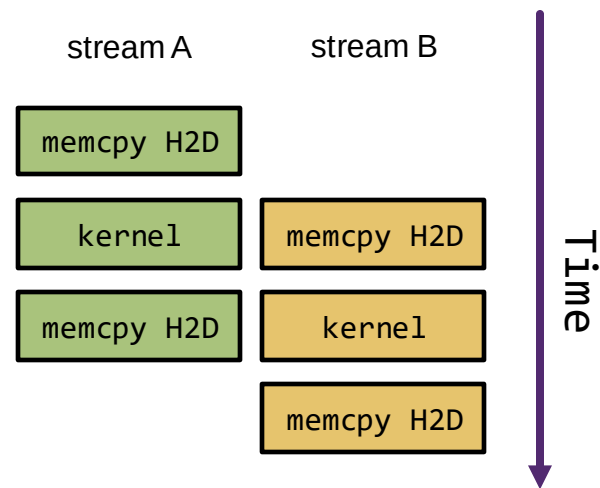


cudaMemcpy是同步计算，无法充分调度

通过 Stream 并发

- 实现并发：
 - 多个 stream
 - 足够的底层硬件资源（寄存器、共享内存等）
 - 采用异步内存传输 `cudaMemcpyAsync`

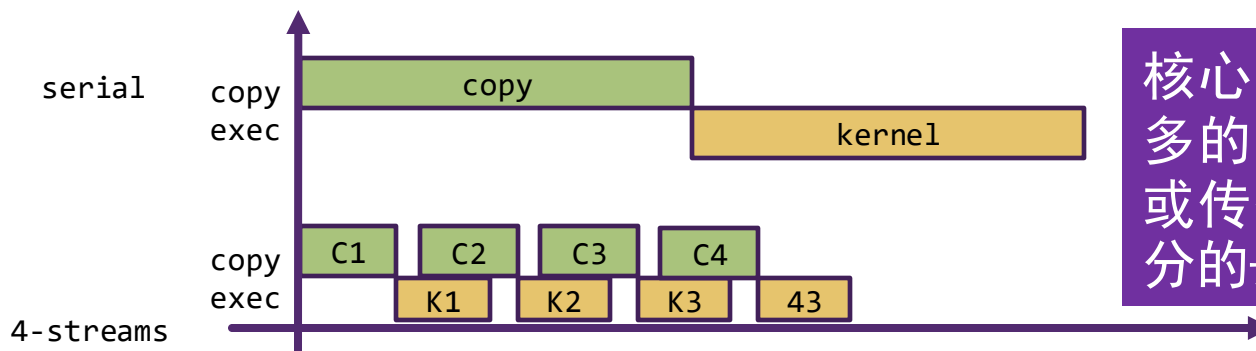
```
cudaStream_t stream[2];
cudaStreamCreate(&stream[0]);
cudaStreamCreate(&stream[1]);
cudaMallocHost(&hostPtr, 2 * size); // page
locked mem)
for (int i = 0; i < 2; ++i) {
    // async memcpy
    cudaMemcpyAsync(/*...*/, cudaMemcpyHost
ToDevice, stream[i]);
    kernel<<<100,512,0,stream[i]>>>(/*...*/
);
    cudaMemcpyAsync(/*...*/, cudaMemcpyDevi
ceToHost, stream[i]);
}
cudaStreamDestroy(stream[0]);
cudaStreamDestroy(stream[1]);
```



数据传输的优化

- 计算和数据传输优化：
 - 对数据进行划分 (partition)
 - 通过多个 stream 并发执行
 - Fermi 架构有1个计算队列、2个拷贝队列 (H2D、D2H)
- 示例 → 从 CPU 拷贝数据到 GPU、Kernel 计算

```
size = N * sizeof(float) / nStreams;  
for (i = 0; i < nStreams; i++) {  
    offset = i * N / nStreams;  
    cudaMemcpyAsync(a_d+offset, a_h+offset, size, dir, stream[i]);  
    kernel<<<N / (nThreads * nStreams), nThreads, 0, stream[i]>>>(a_d + offset);  
}
```



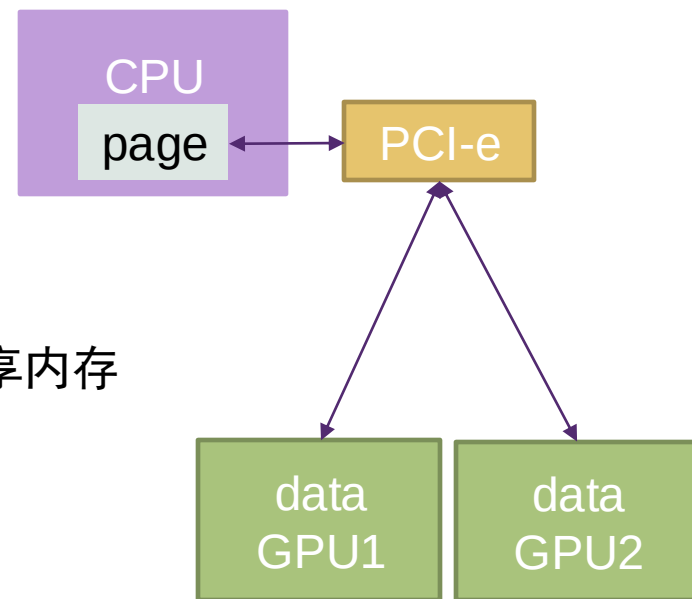
核心：给硬件提供更多的并发操作（计算或传输），让硬件充分的去调度→流水线

固定内存 – Pinned memory

- 固定内存（pinned memory）或页锁定内存（page-locked memory）
 - 页锁定内存将主存上的一个页**直接映射**到 GPU
 - 访存时，直接通过 PCI-e 访问主存
 - 无需通过 `cudaMemcpy` 显式进行数据传输
 - 可以获得 host 与 device 之间很高的带宽（~12 GB/s on PCI-e x16 gen3）

- 注意事项：

- 分配过多页锁定内存会**降低系统整体性能**
- 仅针对不能被重用的数据
- 如果数据可以重用，尽量放在全局内存或共享内存



固定内存– Pinned memory

- 固定内存使用：
 - 在设备上启动固定内存模式： `cudaSetDeviceFlags`
 - 在 host 端进行内存显式分配： `cudaHostAlloc()`
 - 获得 host 端内存对应 device 端指针： `cudaHostGetDevicePointer()`
 - 方便 host 端和 device 端数据实现同步
- 示例：
 - 其中 `host_ptr` 和 `dev_ptr` 指向同一块内存空间

```
int main() {  
    float *host_ptr, *dev_ptr;  
    cudaSetDeviceFlags(cudaDeviceMapHost);  
    cudaHostAlloc(&host_ptr, size, cudaHostAllocMapped);  
    cudaHostGetDevicePointer(&dev_ptr, host_ptr, 0);  
    example_pinned<<<gridSize, blockSize>>>(dev_ptr);  
}
```

全局内存优化

合并访存 - Coalesced Memory Access

- 全局内存的访问是影响性能关键因素之一
 - 相比于共享内存：延迟高、带宽低
 - 需尽可能提高访问效率，并降低访问量

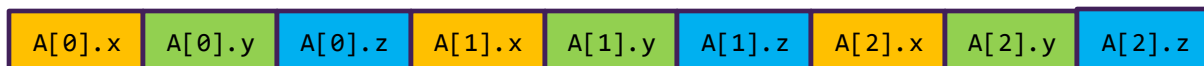
优化策略

- 合并访存 – Coalesced memory access
 - 当 warp 内所有线程执行 load 指令时，硬件会检测线程是否访问连续地址
 - 如果连续：硬件会自动合并 warp 内线程内存访问
- 在全局内存上访存的最小粒度为 sector (32Byte)
 - 如果没有合并访存，一个线程很难满足 sector 的粒度需求
 - 合并访存是提升 GPU 访存性能的关键

AoS 和 SoA 对比

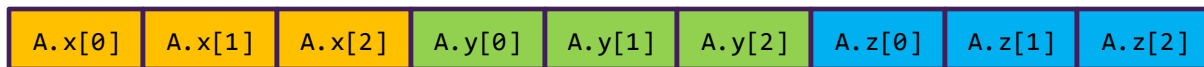
- AoS (Array of Structures) 组织:

```
struct Point { double x; double y; double z; } A[N];
```



- SoA (Structure of Arrays) 组织:

```
struct PointList { double *x; double *y; double *z; } A;
```



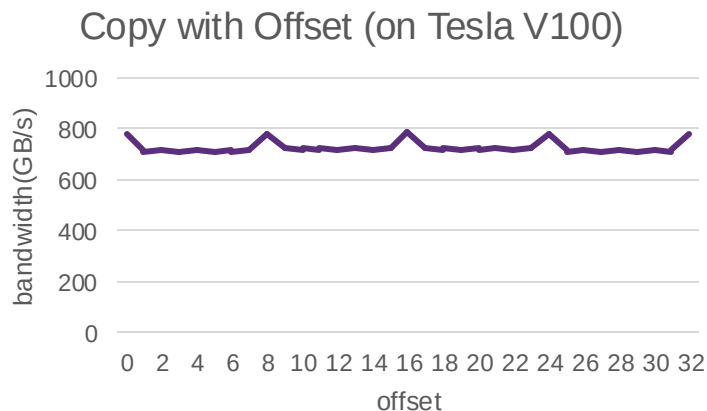
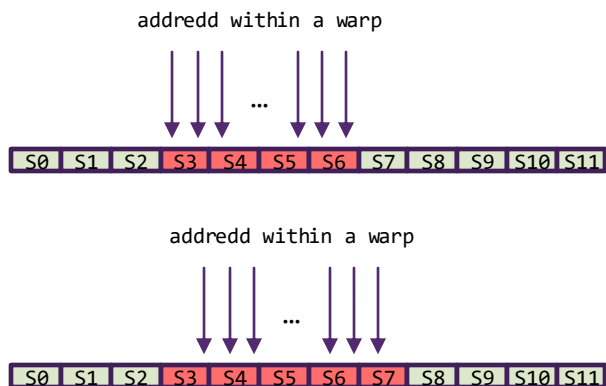
- 如果按照每个线程x,y,z成员访问, GPU 上 SoA 更合适
 - 当线程访问某一组 Point 的成员变量 x 时
 - Warp 每次访问是一段连续内存: 内存合并
 - 与 CPU 上每个线程连续访问恰好相反: 局部性更好

访存对齐

- 访存对齐：
 - 假设 warp 内所有线程内存访问连续
 - 内存的最小访问粒度为 sector (32bytes)
 - 那么访存的地址会影响访问的 sector 个数

```
__global__ void offsetCopy(float *odata, float *idata, int offset) {  
    int xid = blockIdx.x * blockDim.x + threadIdx.x + offset;  
    odata[xid] = idata[xid];  
}
```

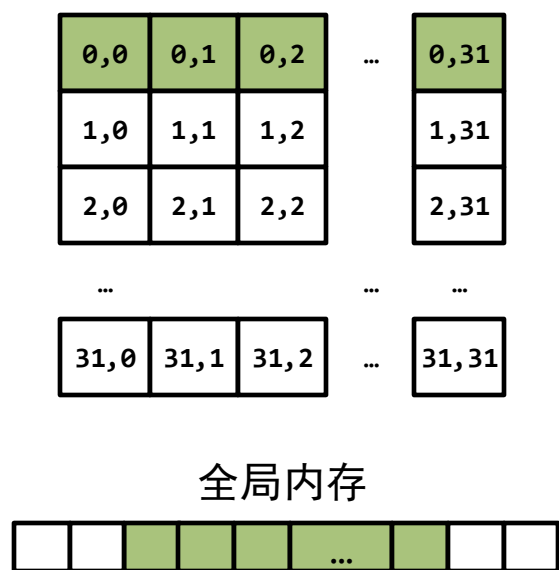
- 例子：在对齐 sector 时，访问4个 sector；不对齐时，访问5个 sector
- 访问次数会影响性能



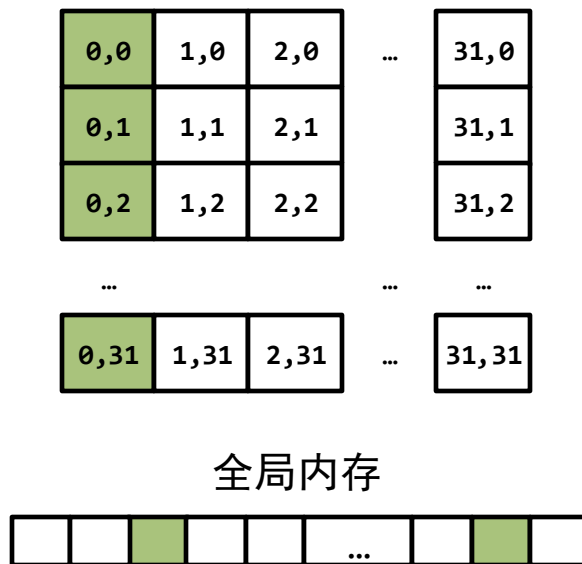
全局内存优化例子

优化例子-1：矩阵转置

- 将一个比较小（不超过几个KB）的矩阵进行转置
- 简单实现：
 - 一个线程读写一个元素
 - 读的时候访问连续内存，内存访问被合并
 - 写的时候不连续，无法合并访存，性能下降严重



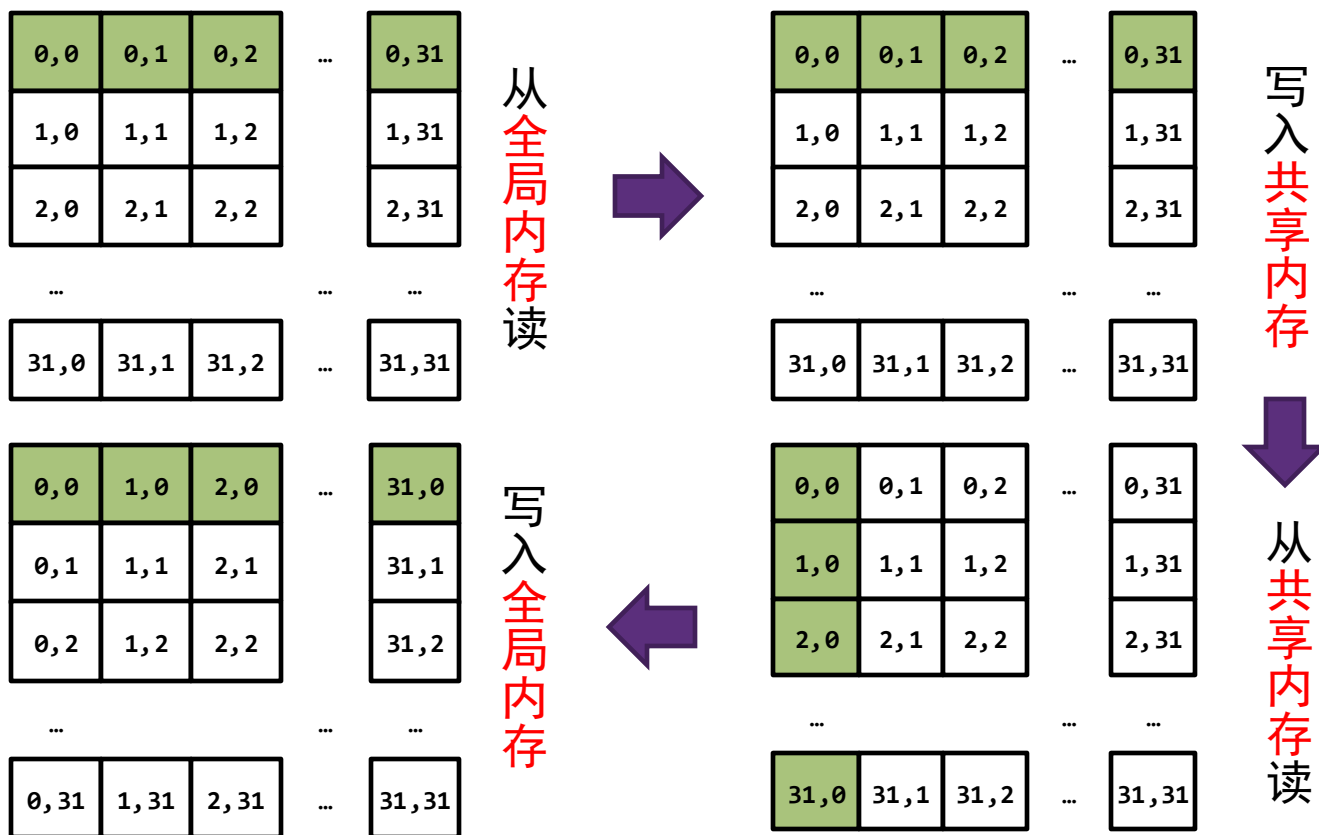
读过程连续



写过程不连续

优化例子-1：矩阵转置

- 优化策略：
 - 使用共享内存做缓存
 - 对全局内存的写可以合并



但可能仍有问题（后面会详细讲）

优化例子-2: APSP - All Pairs Shortest Path

- 使用 $n*n$ 个线程进行计算, 使用全局内存存储结果矩阵 D
- 每个 kernel 计算 $n*n$ 个元素
- 其中每个线程需要 **6次全局内存访问**

```
__global__ void FW_APSP(int k, int D[n][n]) {
    int i = threadIdx.x;
    int j = threadIdx.y;
    if (D[i][j] > D[i][k] + D[k][j]) {
        D[i][j] = D[i][k] + D[k][j];
    }
}

int main() {
    // ...
    dim3 threadsPerBlock(n, n);
    for (int k = 0; k < n, k++) {
        FW_APSP<<<1, threadsPerBlock>>>(k, D);
    }
}
```

优化例子-2: APSP

- 使用 shared memory 缓存中间结果
- 每个线程只进行 2次全局内存访问（6次 → 2次）

```
__global__ void FW_APSP(int k, int D[n][n]) {
    extern __shared__ int S[];
    int i = threadIdx.x;
    int j = threadIdx.y;
    S[i][j] = D[i][j]; // move data to shared memory
    __syncthreads();
    // do computation
    if (S[i][j] > S[i][k] + S[k][j]) {
        D[i][j] = S[i][k] + S[k][j];
    }
}

int main() {
    dim3 threadsPerBlock(n, n);
    for (int k = 0; k < n, k++)
        FW_APSP<<<1, threadsPerBlock, n * n * sizeof(int)>>>(k, D);
}
```

优化例子-3：矩阵乘法

- 矩阵乘法：

```
for (int i = 0; i < N; i++)  
    for (int j = 0; j < N; j++) {  
        auto tmp = C[i][j];  
        for (int k = 0; k < N; k++)  
            tmp += A[i][k] * B[k][j];  
        C[i][j] = tmp;  
    }
```

- 计算访存比（Compute to Global Memory Access - CGMA）

- **计算**：1次乘法 + 1次加法 = 2次
- **访存**：2次读
- CGMA = 1

- 以 V100 为例，V100 中计算和访存的性能差约为 **70倍**

- 需要提高 CGMA 到 **70左右**才能够发挥 V100 的计算性能

如果放入共享内存？

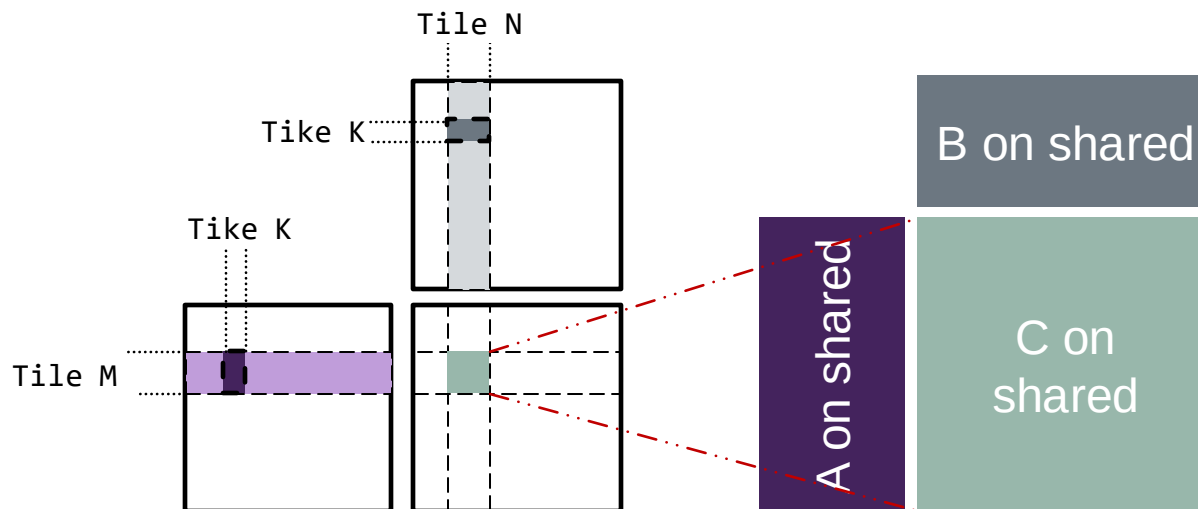
- 矩阵乘法：

```
for (int i = 0; i < N; i++)  
    for (int j = 0; j < N; j++) {  
        auto tmp = C[i][j];  
        for (int k = 0; k < N; k++)  
            tmp += A[i][k] * B[k][j];  
        C[i][j] = tmp;  
    }
```

- 如果把所有访存全部预取到共享内存
 - 共享内存具有和 cache 一样更快的性能
 - 那么 CGMA=N（每行或每列被重用，N倍重用）
 - N够大时，访存就不是瓶颈
- 共享内存没有那么大！
 - 数据要存 $3N^2$ ，N=1024 时，需要12MB
 - 典型共享内存 128KB/SM

分块算法 - Blocking (Tiling) Algorithm

- 分块算法核心：
 - 将任务拆分为子问题
 - 每一个子问题的访存操作会被限定在较小范围
 - 每个子问题的访存可以利用更快存储加速
- 下图是 GPU 中对矩阵分块后的存储方式



分块算法

- 通过分块算法重写矩阵乘的循环

```
for (int i0 = 0; i0 < M; i0 += TILE_M)
    for (int j0 = 0; j0 < N; j0 += TILE_N)
        for (int k0 = 0; k0 < K; k0 += TILE_K) {
            // load data to shared memory
            // ...
            for (int i = 0; i < TILE_M; i++)
                for (int j = 0; j < TILE_N; j++)
                    for (int k = 0; k < TILE_K; k++)
                        bufC[i][j] += bufA[i][k] * bufB[k][j];
        }
```

全部的访存个数 =
 $2 \times (\text{TILE_SIZE})^2$
全部的计算个数 =
 $2 \times (\text{TILE_SIZE})^3$

- 找到合适的 TILE 大小，使需要的数据都可以放在共享内存中
- 数据可被重用大致 TILE_M 或 TILE_N 次
 - 计算： $2MNK$ （乘加） 访存： $MK+NK$ (从全局内存读到共享内存)
- 假设有 48KB 共享内存（假设方形, $\text{TILE_M}=\text{TILE_N}=\text{TILE_K}$)
 - $\text{Max tile size} = (48\text{KB}/4\text{B}/3) ^ (1/2) = 64$
 - $\text{CGMA} = 64$

分块算法

- 分块算法：
 - 将输入切成若干块放入共享内存或 cache 中
 - 增加数据重用次数，减少全局访问
- 更高的 CGMA 并不一定永远带来更好的性能
 - CGMA 应该足够大以掩盖访存时间
 - 并不是越大越好
 - 分块算法有时候会增加开销
 - 分块大小需要经过精细调整

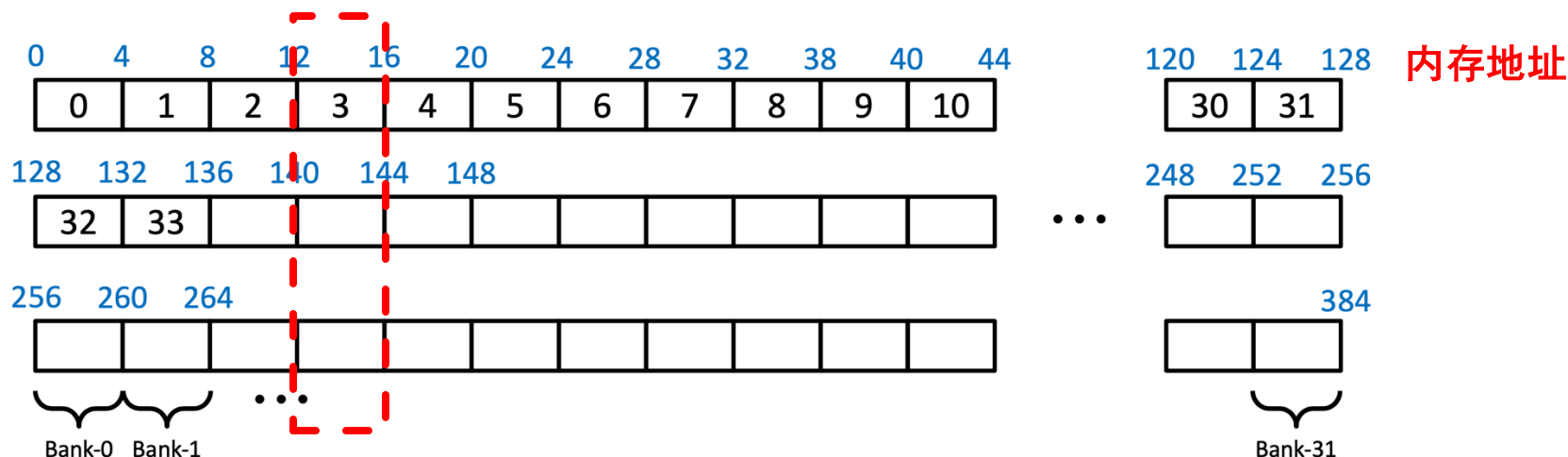
共享内存优化

共享内存

- 共享内存
 - 每个 SM 上的高速存储
 - 需要用户显示管理
- 相比于全局内存
 - 20~40 倍的延迟降低
 - ~15 倍的带宽提升
 - 访存粒度 4Bytes
- 共享内存用于
 - 频繁重复访问的数据，减少全局内存访问
 - 线程块间线程通信

共享内存组织 - Bank

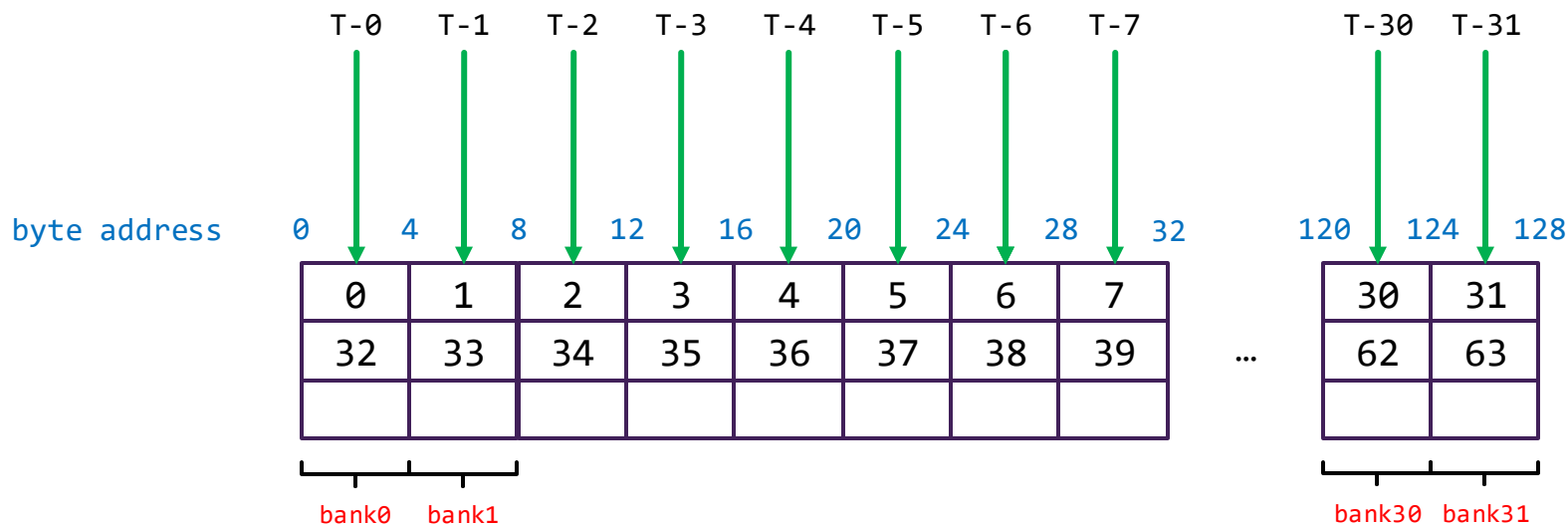
- 为了支持多个线程并发访问：
 - 共享内存被划分成一系列连续 bank
 - 每个 bank 宽度 4Bytes (32-bit)
- 线程提供的内存访问地址
 - 硬件确定地址属于哪个 bank
 - 读操作：在线程之间分配请求的数据，允许广播
 - 写操作：多个线程写入相同的地址，一个“成功”



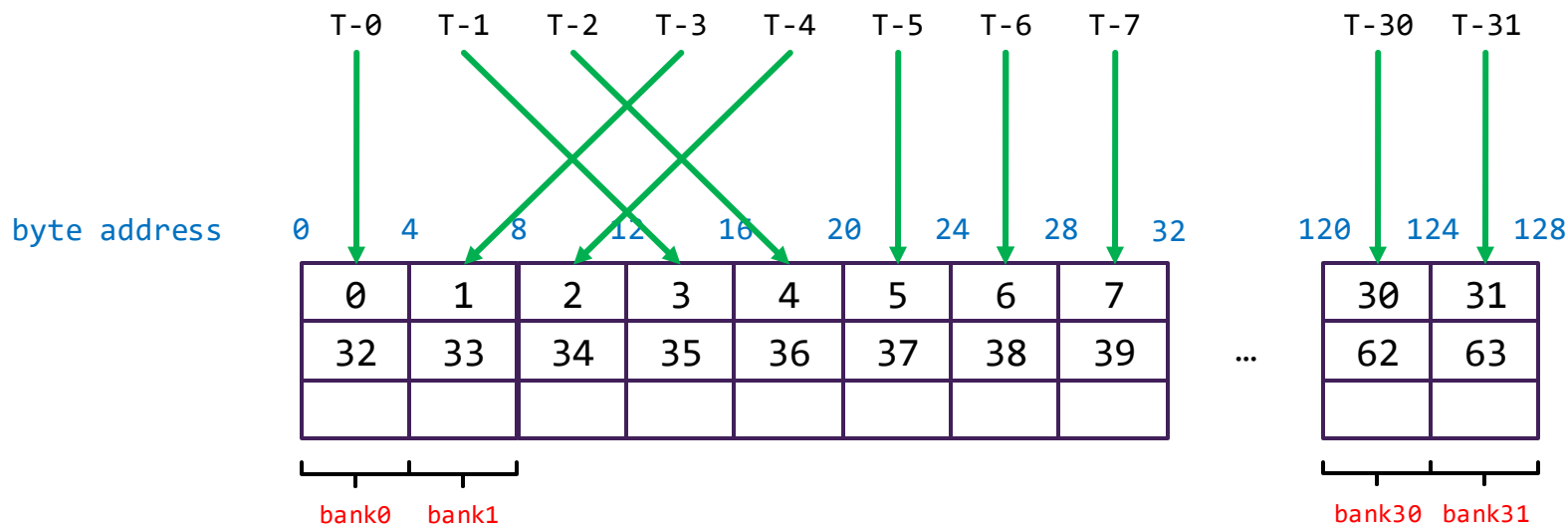
Bank 冲突 – Bank Conflict

- 在同一周期每个 bank 只能服务一个请求
 - 共享内存可以最多支持最大 bank 数内存请求
 - 多个内存请求访问同一个 bank 时，会导致 bank conflict
- Bank conflict
 - 在同一个 warp 内，访问同一个 bank 内的不同 4-byte（不同行）
 - 发生 bank conflict 会严重影响共享内存访问性能（顺序执行）
- N-way bank conflict
 - 一个 warp 内 n 个线程冲突
 - 增加 n-1 倍访问延迟
 - 最坏情况：32-way bank conflict（全部线程）
- 不存在 bank conflict 的情况
 - 不同线程访问同一个 4-byte 的不同 byte
 - 不同线程访问相同 4-byte 内容
 - 不同 warp 线程访问同一个 bank（分别调度）

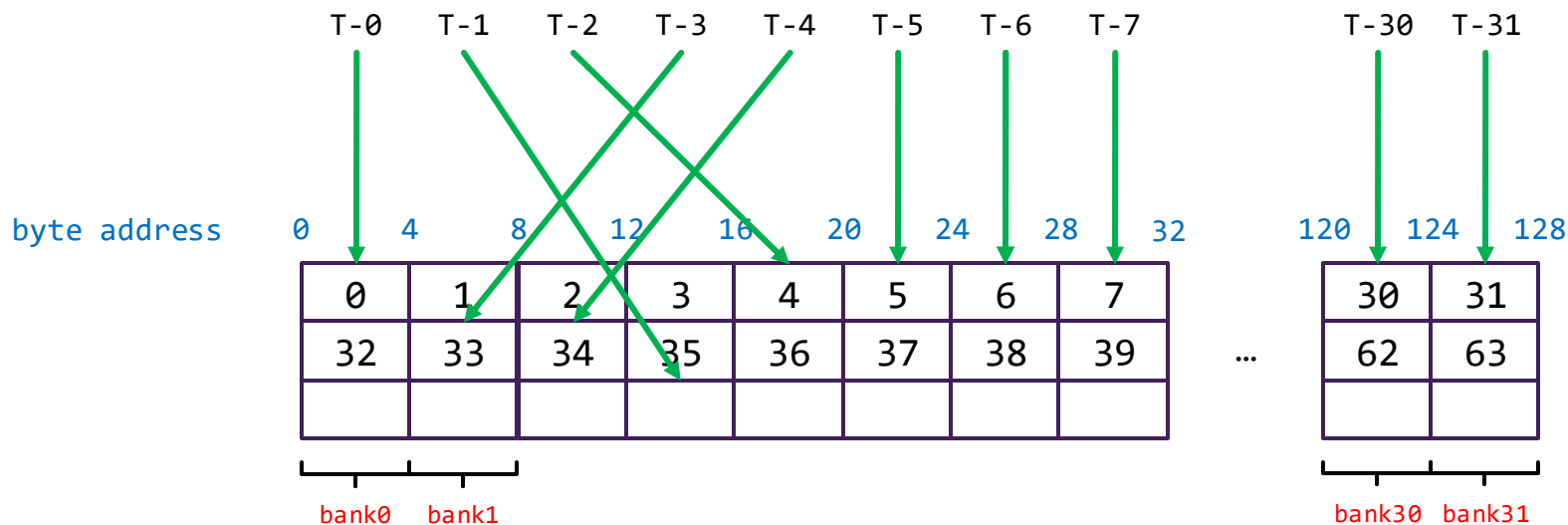
没有 Bank Conflict



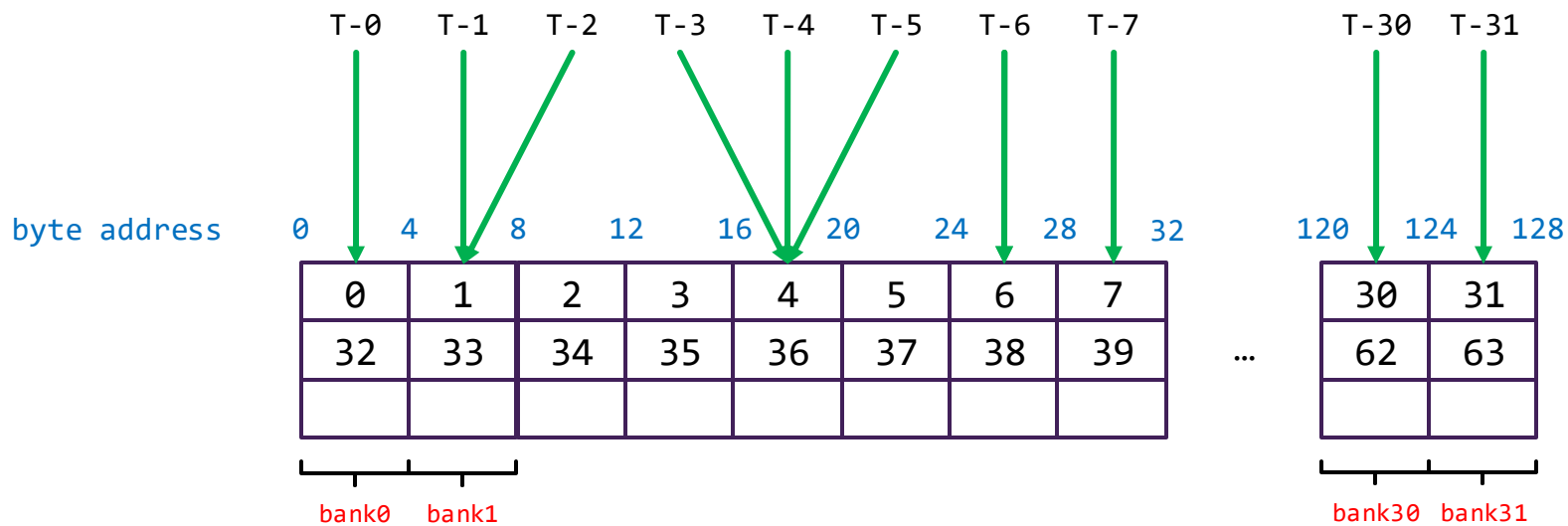
没有 Bank Conflict



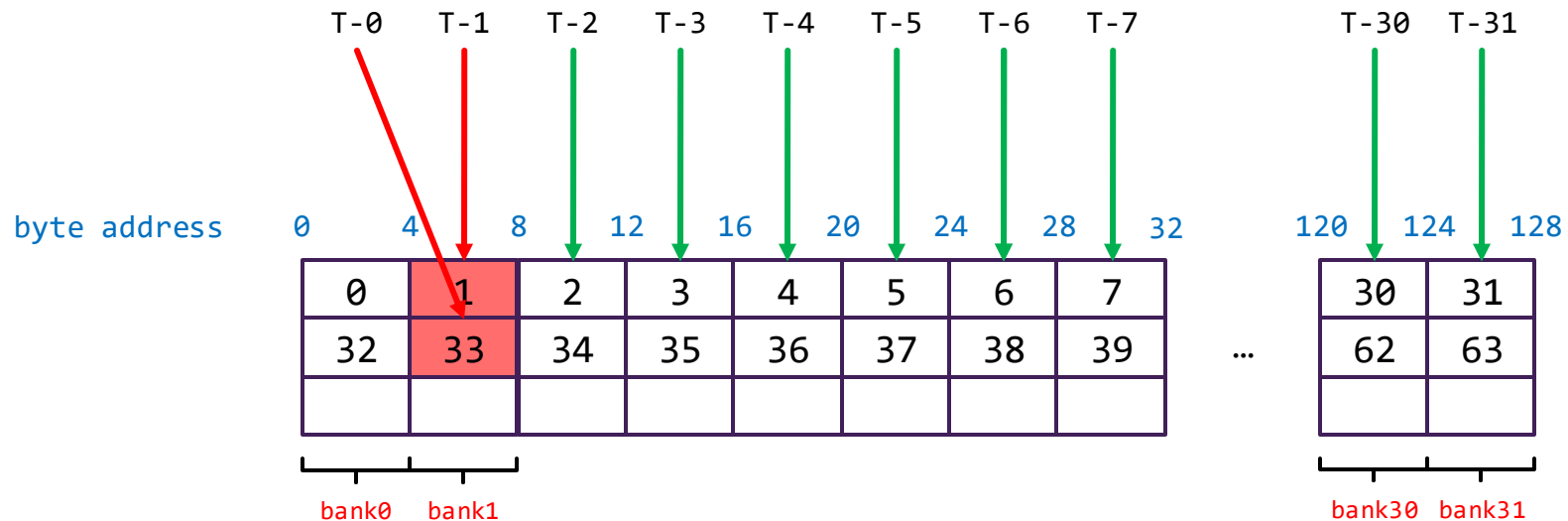
没有 Bank Conflict



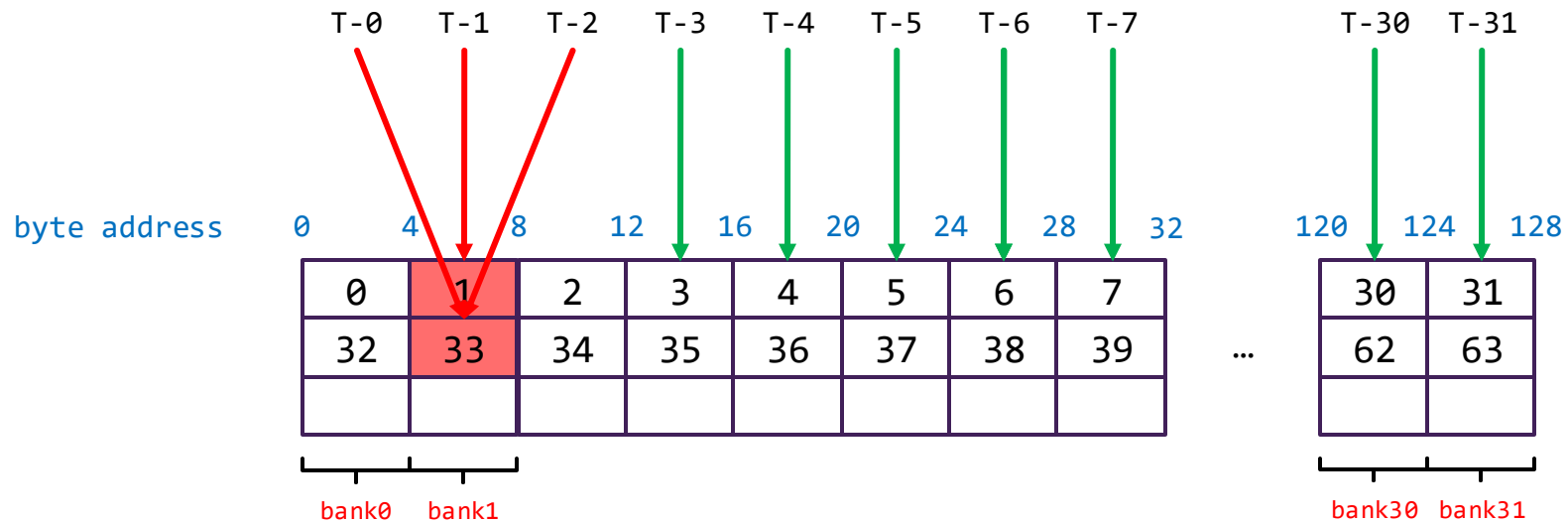
没有 Bank Conflict



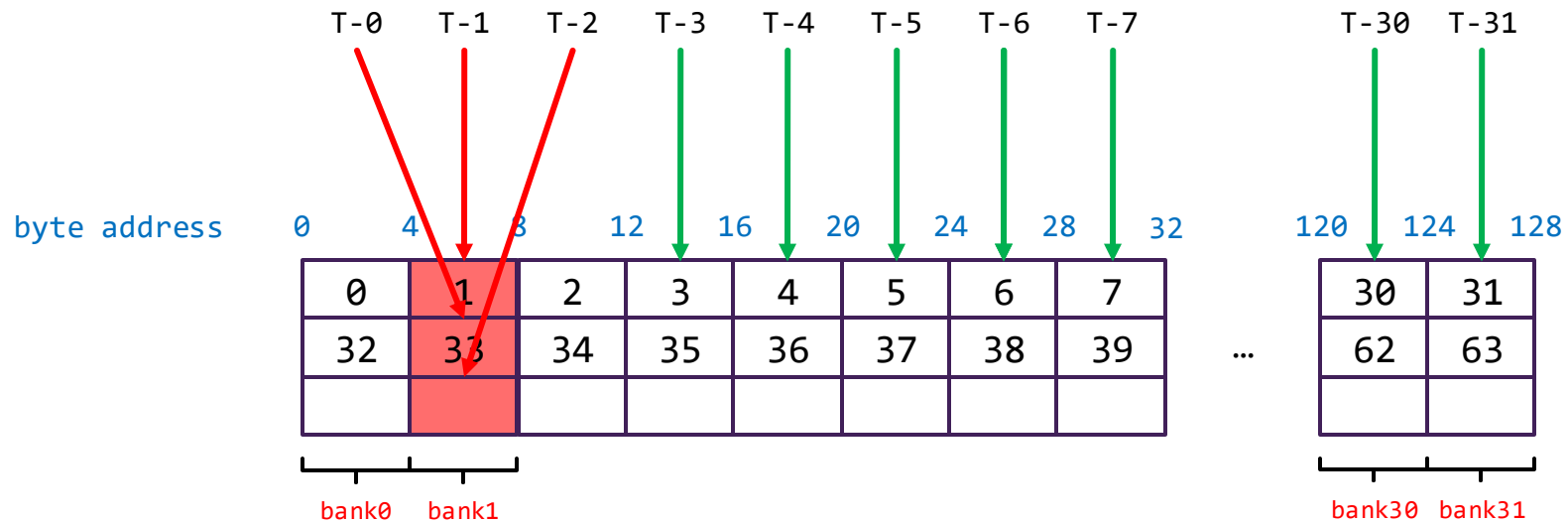
2-way Bank Conflict



2-way Bank Conflict



3-way Bank Conflict



例子-1: 矩阵转置

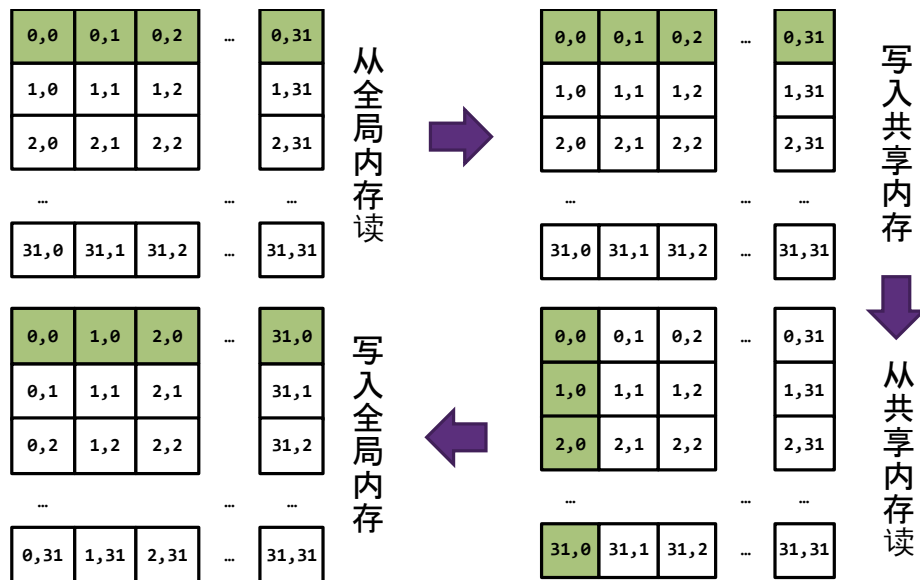
■ 矩阵转置

- 通过 shared memory 合并访存操作

- 每个 block 有 32×32 个线程
- Block 内存储 32×32 的 FP32

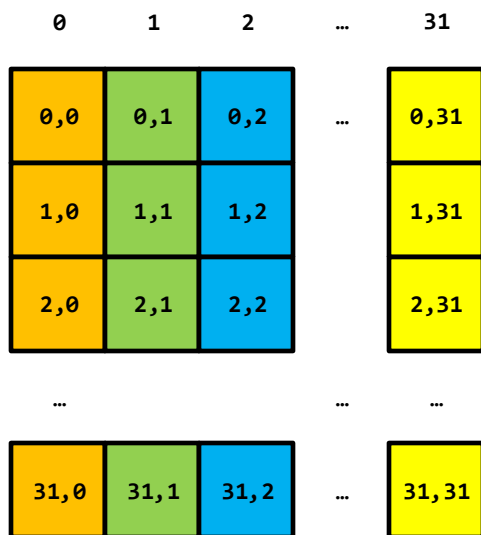
■ 基本的实现:

- 一个 warp 从全局主存读取一行
- 写到共享内存中的一行
- 同步 block
- 一个 warp 从共享内存读取一列
- 写到全局内存中的一行



例子-1：存在问题

- 32 * 32 的 shared memory
- `__shared__ float smem[32][32]`
- 行访问：没有 bank conflict
- 列访问：32-way bank conflict
- 严重影响共享内存的访问性能
- 右图中不同颜色表示不同 bank



例子-1: 解决方案

- 解决方案:
 - 共享内存填充
 - `__shared__ float smem[32][33]`
 - 在每一行做一次填充
 - 这样每一列访问 warp 内访问不同 bank
 - 行访问和列访问均没有 bank conflict



共享内存 - 小结

- 共享内存是一块巨大的资源
 - 非常高的带宽 ~14TB/s
 - 相比于全局内存非常低的带宽
 - 用户可控的 scratch-pad 内存
 - 在 V100 上最大可配置到 96KB
- 查找性能问题
 - Bank conflict 会导致延迟增高，带宽降低
 - 使用 profile 工具检查 conflict 事件

总结

- GPU 执行模型
- GPU 存储层次
- CUDA 程序优化
 - 数据传输优化
 - 全局内存优化
 - 共享内存优化

小作业-7

- 访存优化是CUDA性能优化中的重要部分
- 使用不同的参数执行给定的两个程序：test_gmem, test_smem
- 阅读程序，比较程序在不同条件下的执行性能，并分析原因
- 目的：
 - 了解CUDA编程中global memory和shared memory的使用方法
 - 理解影响GPU访存性能的主要因素
- 详情见网络学堂发布

大作业二：GPU 全源最短路

- 任务：GPU 加速的 Floyd-Warshall 算法
- 目标：
 - 熟悉 CUDA 编程
 - 熟悉针对 GPU 体系结构的优化和“分块”优化方法
- 详见实验指导书
- 时间：3周时间